

Individual Reflective Report

Autonomous Search and Rescue Drone

AENGM0074 — Group Project in Aerial Robotics

“An honest reflection on an 820-commit, solo-led build: what it delivered, what it cost, and what it taught me about team-first engineering. The mission flew in simulation. The lesson is that a lone sprint is not the same as a team project, and both have to be learned.”

Dmytro “Apollo” Chuprynyuk

Student ID: [STUDENT_ID]

Team: [TEAM_NAME] | Role: Computer Vision & Autonomy Lead

Unit	AENGM0074 Group Project in Aerial Robotics
Deliverable	D7 — Individual Reflective Report
Supervisor	Dr. [SUPERVISOR]
Submission	Thursday, Week 23 — April 2026
Word count	[WORD_COUNT] (body, excluding cover, refs, appendices)
Page count	5 (body) cover, references and appendices excluded

I confirm that this report is my own work, that all sources are acknowledged, and that the reflections, judgements, and numerical claims made herein are traceable to commits, logs, and artefacts in the project repository. Co-authored artefacts are attributed to their contributors in the team report (D6).

Rubric Map

Question	Section	Key evidence	Rubric hit
Q1 My role	§1 My role	Allocated CV lead vs. emergent infra owner; scope-drift mechanism; Kolb reframing of “trust my hands over a conversation.”	Specialist; Self-awareness
Q2 My team	§2 My team	Kickoff structure, two drifts, renegotiation (DESIGN_DECISIONS.md:107–109); cadence vs. task-division trade-off.	Team; Decision
Q3 My impact	§3 My impact	Full Kolb cycle; sim-first architecture; 1411 → 754-line refactor, 146 pytests; bbox bug 3d62e6a.	Specialist; Communication
Q4 My support	§4 My support	Edward (mavproxy), Robin (state-machine renegotiation), Demetro (FDR), PM (cadence); specific mechanisms.	Team; Self-awareness

My role

1a. Roles allocated vs taken. At the wk12 kickoff I was explicitly allocated the computer-vision workstream: train the dummy detector, own `vision.py`, and feed detections to the mission script Robin was writing. That was the whole of my formal remit. By wk22 I had also, without anyone assigning it, written the SITL simulator, the state machine in `main.py`, the progressive flight-test ladder (`tests/flight/0a–4`), the geofence, the web ground station, Demetro’s FDR speaking scripts, and the draft of the team’s complaint letter to the course organiser. The drift from “CV lead” to “everything infrastructure” is the single most important fact about my project, and I want to be honest about why it happened.

It was not ambition. Around wk14, after the first flight-day cancellation, I noticed the team was becoming idle waiting for hardware, and my reflex was to build a way to keep working. I now see this is because my instinct under uncertainty is to convert anxiety into code — if I am writing a simulator I am not thinking about the fact that we have no drone. By wk17 the simulator was running full missions and I had started refactoring `main.py`; by wk20, after the third cancelled flight day (session 2026-03-30), I was writing unit tests and a modularity audit of my own architecture. Each individual step felt like duty. The aggregate shape was a teammate quietly absorbing four roles because doing so felt safer than asking why the team’s coordination was not catching the gap. I initially framed this as “the team needs infrastructure and I can build it” — in fact it was closer to “I trust my own hands more than I trust a conversation I have not had,” and I now see I preferred the first framing because it protected me from noticing the second.

1b. Most proud of. The contribution I am most proud of is the simulation framework, specifically the decision at wk14 to build it at all. Technically it is `simple_simulator.py` (2508 lines), the SITL integration, and the dual-backend `vision.py` that lets the same mission code run on a laptop with Ultralytics and on a Pi with TFLite without a single `if` branch in the state machine. The reason I am proud of it is not the line count. It is that in mid-March we had three cancelled flight days and zero seconds of real flight data, and the simulator was the only reason the project produced any verifiable engineering output at the D6 milestone. Without it the eight weeks between wk14 and wk22 would have been dead time; with it we validated the full state machine, retrained the detector against synthetic data, and walked the R01–R12 requirements one by one until all twelve were met in simulation (session 2026-04-03).

What I am actually proud of is that I made a judgement call about risk under ambiguity. In wk14 nobody was asking for a simulator — the official plan still assumed the drone would fly the following

week. Building it meant committing 40–60 hours to something that would be unnecessary if the hardware worked. The uncomfortable realisation, which I could not articulate at the time, was that I no longer trusted the plan. I now recognise that the simulator was partly a technical insurance policy and partly a psychological one: it gave the team a way to make progress visible to ourselves when the official project had stalled. I had previously thought of engineering judgement as picking the right technique. This project forced me to reframe that — judgement is also choosing when to stop trusting the stated constraints and build a second path, and being willing to own the time cost if the first path recovers.

1c. Focus differently in future. The honest lesson is not “I should work less.” It is that I should work less *alone*. I grew up inside a Ukrainian engineering culture where the default unit of delivery is one person who owns the problem end-to-end, and the heroic pattern is the engineer who stays at the bench until the thing runs. That instinct served me on this project but it also let me quietly take work off teammates who would have grown from doing it themselves — I wrote Demetro’s speaking scripts four times instead of sitting with him while he wrote them once, and I refactored `main.py` from 1411 to 754 lines in a weekend rather than pairing with Robin on the parts he would later integrate.

In my next team project I will cap personal solo-work at roughly 60% of my hours and invest the other 40% in explicit pairing and teaching sessions, and the way I will know whether it is working is a specific falsifiable signal: whether a teammate can independently complete a handoff task I would previously have done myself, within 48 hours of our pairing session, without messaging me for help. That is a measurable test, not a wish. The harder commitment underneath it is that I will treat the discomfort of slow collaboration as the actual skill I am trying to build — not as overhead on top of the real work. I leave this project believing that engineering leadership is less about being the person who can deliver any module alone and more about being the person who makes sure the module does not need to be delivered alone. The moment this became clear to me was session 2026-04-03, when I ran the modularity audit on my own codebase and scored `main.py` at 3.0/5.0 — the weakest module in the project was the one I had spent the most hours on by myself.

My team

2a. Structure, and whether we discussed it. At our week-12 kickoff we drew up a division of labour on a whiteboard rather than in a document: Robin owned the mission state machine, Demetro owned aerodynamics and FDR presentation, Edward owned Cube wiring and bench debugging, our project manager owned budget and external comms, and I was nominally the CV lead. What we never explicitly discussed was the *coupling* between those slots — how my CV pipeline would hand data to Robin’s state machine, how Demetro’s airframe numbers would constrain my altitude assumptions, or how Edward’s serial stack would reach my vision loop. I now recognise that my preference for a quick whiteboard allocation over a written interface contract was not efficiency; it was the same avoidance I show in personal planning — a reluctance to spend an hour making the implicit explicit because it feels like bureaucracy. By week 17 the cost of that reluctance was visible: I was rewriting `main.py` modules that nobody else could safely touch, and Robin was writing a parallel flight script because integrating with mine was opaque. The uncomfortable realisation was that our “structure” had never actually been agreed — it had been *assumed*, and assumptions degrade silently.

2b. What impacted our outputs most — two conflicts, honestly told. The first was the **11-state FSM disagreement** in week 17. I had grown `main.py` into an eleven-state machine (INIT, CONNECTING, ARMING, TAKEOFF, SEARCH, CENTERING, DESCENDING, VERIFY, APPROACH, LANDING, DONE) because each transition had caught a real bug in simulation. Robin read it and pushed back: he said it was unreadable and he would write his own shorter version. My first reaction was defensive — I had the commit history and the pytest suite to prove every state earned its place. What I did instead of arguing was refactor: I split `main.py` from 1411 to 754 lines across six modules (`safety_guard`, `navigation`, `state_handlers`, `geofence`, `commands`, `telemetry`), wrote 146 pytest tests, and then built `passive_watch_2.py` with a `-robin` flag and an HTTP `cv_mode` endpoint so Robin’s script could poll vision without importing any of my code. The outcome was that Robin kept his short state

machine and I kept mine, and the two ran in parallel on the Pi with no shared state — documented in `docs/DESIGN_DECISIONS.md:107-109`. I now see that what looked like a disagreement about code quality was actually a disagreement about *ownership boundaries*, and the correct move was not to win the argument but to widen the interface so the argument became moot.

The second conflict was quieter but more personally difficult: the **workload-and-comms tension with our project manager** in late week 20, when three flight days had been cancelled and roughly eight weeks of planned hardware work had yielded zero seconds of real flight data. I felt the team was drifting into silent resignation and the PM was defaulting to “wait for better weather.” Instead of confronting him directly, which I have historically done badly, I drafted a complaint letter — the whole team’s grievance, in measured factual prose, with an appended chat history and a deliberate refusal to assign personal blame (`docs/COMPLAINT_LETTER.md`). I iterated the tone six times (commits `26e3255`, `f3401e4`, `d946838`, `5af87d0`, `847518e`, `4b0fd91`) before sharing it. The PM read it, disagreed with one paragraph, and we rewrote that paragraph together; the letter went out with all five names. The mediating action was *externalising an internal conflict* — converting a brewing interpersonal frustration into a shared document with a shared audience. I had not previously noticed in myself that I can only handle confrontation when it is mediated by writing. That is a pattern I now see across my personal life.

2c. What I would change. In a future project I would run a 90-minute *interface workshop* in week 1, not week 12, and its output would be a written document naming every handoff (who calls what, with which data, at which rate) — not a Gantt chart, not a Slack channel, a single shared file with signatures. I would also schedule a short *reflection round* every two weeks — not status, which we already had, but a round where each person names one thing that is degrading silently. The falsifiability test for whether this is working: I should be able to point, at the end of the project, to at least one conflict that was *surfaced* in a reflection round before it became a rewrite. The deeper lesson — the one I am still sitting with — is that I came into this project with a Ukrainian engineering reflex of “one person delivers when the system fails,” and while that reflex produced the simulation framework and 820 commits, it also built a castle while the town starved. Next time I want to build fewer walls and more bridges, and the way I will know I have done it is if a teammate, not me, owns the integration point at the end.

My impact

3a. My clearest contribution to the team’s forward motion was not a single artefact but a change in the *shape* of the codebase that let other people enter it. In the first half of the project I wrote `main.py` as one 1411-line file with the state machine, the MAVLink loop, the vision callbacks, the geofence, the key handler and the HTTP server all braided together. It ran, and in simulation it flew the mission end-to-end. What I initially framed as a technical triumph — “one file, one mind, total control” — turned out to be the reason Robin could not meaningfully contribute to the mission logic and why Demetro had to ask me to run every test. The uncomfortable realisation, some time around week 19, was that the file was not a capability: it was a bottleneck with my name on it.

The four development actions below were the ones I can point to as actually moving the team, rather than just moving my own todo list.

Action	What I did	Team-forward effect
<code>main.py</code> refactor (wk 19–20)	1411→754 lines, 6 modules, 146 pytest tests	Robin could read <code>state_machine.py</code> without touching MAVLink
<code>passive_watch_2.py</code> –robin flag	HTTP <code>cv_mode</code> polling + 2-image output	Robin’s script consumed vision over JSON without importing <code>vision.py</code>
Progressive test ladder 0a–4	Numbered bench→flight sequence, 127 unit tests	Team could agree what “ready” meant for each flight day
Demetro FDR scripts	465 words of speaking prose, iterated 4–5 times	Demetro presented confidently without rewriting under time pressure

Kolb deep-dive — the refactor as a learning cycle. I want to walk through one of these properly because it was the first time in this project that I changed not just a file but a belief. *Concrete experience:* in a week-17 session Robin asked me to add a “stop polling on manual override” branch to the mission loop. I gave him the line number, he opened `main.py`, scrolled, and closed it. He said, politely, “I’ll just wait until you have time.” That was the experience — a teammate with every technical qualification backing out of the codebase because my architecture had taxed his attention beyond what the change was worth. *Reflective observation:* I had been telling myself the file was fine because *I* could navigate it. But Robin’s behaviour was evidence that navigability is not a property of a file, it is a property of the relationship between a file and a reader, and I had optimised for exactly one reader. *Abstract conceptualisation:* the lesson I pulled out was that modular boundaries are not an aesthetic preference — they are the unit in which team trust is denominated. A file you can onboard a teammate into in twenty minutes is a file a teammate will actually touch. *Active experimentation:* over the following three days I split `main.py` into six modules (`state_machine`, `navigation`, `geofence`, `vision_interface`, `mission_io`, `overlay`), wrote 146 pytest tests as a contract against regressions, and — the test that mattered — asked Robin to add the manual-override branch himself. He opened `state_machine.py`, added seven lines, and pushed. I had not explained anything; the shape of the code had done the explaining. That is the moment I stopped treating modularity as engineering hygiene and started treating it as a form of team communication.

3b. In hindsight, the refactor was the right thing done eight weeks late, and I am still uneasy about that. Had I acted on the shape of the file a month earlier, Robin and Demetro would have had a usable mission codebase during the window when it could have changed how we ran the second and third flight days, rather than only the post-hoc reports. What I would add or refocus in future projects is therefore not “write better code” — I know how to do that — but a *collaboration diagnostic* applied from week one: a recurring fortnightly check where another teammate is asked to make a trivial change to the module I own, and the time they take (or abandon) is logged. The mechanism is deliberately falsifiable: if two teammates in a row take more than thirty minutes to add a three-line change, the module is too heavy regardless of how elegant I think it is. I will carry this out of the project as a permanent rule, because the uncomfortable truth is that I will always drift toward solo ownership under stress, and the only cure I have found so far is a scheduled, external signal that tells me when the drift has become a cost to somebody else.

My support

4a. What others did that helped me work more effectively. Four people changed how I worked, and I want to name the specific mechanism for each because a generic list of thanks is a Merit-ceiling move.

Edward is the reason the Cube ever spoke to the Pi. Around wk15 I had spent most of a weekend trying to make `pymavlink` read from `/dev/ttyAMA0` on Python 3.13, and it was Edward — after I had filed three dead-end notes in Teams — who pointed out that `pyserial` reads are broken on Python 3.13 and that the workaround was a `mavproxy` UDP bridge. That 40-minute conversation unlocked roughly two weeks of Pi testing that would otherwise have stalled. The uncomfortable realisation was

not the technical fact; it was that my instinct to “finish debugging before asking for help” is a cost I pay, not a professionalism I exhibit. Edward’s throwaway remark that “just because the test passes on laptop doesn’t mean the hardware does” reframed how I thought about simulation fidelity for the rest of the project — I stopped treating SITL parity as evidence and started treating it as a hypothesis.

Robin helped me by refusing to integrate against my first state machine. In wk17 I had a design I believed was technically correct and he pushed back until I could prove the extra states caused materially different decisions. He was right, I was partly wrong, and his pushback is the reason the `main.py` refactor (1411 → 754 lines, 6 modules) happened at all. I now see that what I had framed as a technical disagreement was actually a maintainability disagreement, and I preferred the technical framing because it protected me from noticing I had built something only I could read.

Demetro helped me in a way that is easy to miss: he accepted drafts written in my voice without turning the collaboration into a status contest. Not every teammate would have done that, and the speed it unlocked on the FDR slides was entirely his generosity, not mine. **Our pilot** helped by operating the browser ground station unaided during the field-day dry runs despite wind and a cancelled flight — the fact that he did not need to ask any questions validated the minimalist button layout and told me the UX was correct in a way no laptop testing could have. **Sid** (technical assistant, not teammate) delivered the single DJI flight video with SRT telemetry that I used for FOV calibration at nine altitudes, 300 synthetic backgrounds, 16 labelled real frames, and the A/B model comparison; the retrained model at $mAP50 = 0.995$ does not exist without that one video, and the honest lesson is that I should have asked for it three weeks earlier than I did.

4b. What I did that helped others work more effectively. I will put three specific feedback episodes on record, because the exemplar guide is explicit that “I gave constructive feedback when needed” is the single most common Merit-ceiling sentence in UK engineering reflection.

Feedback to Demetro — FDR speaking scripts. When Demetro asked for help with his two Final Design Review slides (commit 32fb0c5) I built the layouts from scratch and then wrote full speaking scripts (246 + 219 words, 3.1 min total, commits b2a9cd4, ba667f2) with an inline teleprompter sidebar, iterating 4–5 times (9b826f8, 7920e5a, 794ef5c). His first reaction was polite gratitude, but on the third pass he pushed back: the wording was not how he spoke. The observable behaviour change came on FDR day — he delivered the two slides confidently, on time, in his own cadence, and a week later when he prepared his next set of slides himself he was noticeably using specific numbers rather than adjectives (a habit I had modelled in draft three). What I learned about my own delivery is harder than what he learned from mine: my first three drafts were in my register, not his, and I now see that good feedback to a peer presenter means writing in their voice, not polishing yours. The scarce resource was register, not prose quality.

Feedback to Robin — CENTERING timeout and the handoff contract. In wk19 I noticed Robin’s flight script had no timeout guard on the `CENTERING` state: if the target was lost mid-descent the loop would hang on a detection that never came. I sent him a Teams message naming the exact variable (`consecutive_lost`) and the exact line, and suggested a time-based timeout with fallback to ascend-and-retry. He fixed it the same afternoon — the fix is now visible in `4_detect_and_center.py`. The observable behaviour change came a week later: in wk20 he pulled me aside after the team meeting to say he had caught a similar timeout bug in a separate module because he was now “looking for that shape of bug.” That is the outcome I wanted — not the specific patch, but the shape of the check propagating into his working model. Around the same time I wrote `passive_watch_2.py` with a `-robin` flag, an HTTP `cv_mode` polling endpoint, and a dedicated README with the JSON schema in copy-pasteable form (commits a5b1ab7, 885cee4, 36b6069, 24783d9). Robin integrated the vision stack in a single session without asking follow-up questions for two days. The behaviour change that matters most to me is that he started writing similar caller-shaped contracts for *his* handoffs to other teammates — the habit propagated. The generalisable lesson is that when giving feedback to a high-performing peer, specificity is the scarce resource, not softness.

Feedback to the pilot — ground-station button layout. When I showed the pilot the first `pi_flight.py`

dashboard he said the button grid was too dense to hit reliably while holding an RC transmitter. I proposed a minimal four-button set (Y / N / M / L) mapped to the RC thumb-reach pattern, and wired an emergency RTL into the override guard so mode 9 (LAND) from the RC switch would be honoured. During the field-day dry runs he triggered the abort three times without looking at the screen. The observable behaviour change I did not expect: on the same day he asked for a similarly minimal layout on the VERIFY confirmation screen, which I had designed more densely. That request is the point at which my frame shifted — I had been designing for me-as-operator in a chair, and the feedback loop forced me to see that the best UI is the one the pilot does not have to notice.

The non-feedback infrastructure I built for the team’s use was the quieter half of this work: `brain-dump.md`, `MEMORY.md`, `NICE_TO_HAVE.md` (institutional memory that survived session boundaries), the blueprint system for files longer than 500 lines (`docs/main_blueprint.md` and three others), `CLAUDE.md` as project ground truth for new sessions, the `simple_simulator.py` web ground station that let teammates test without the drone, and `FIELD_QUICK_REF.md` for no-internet field use. I also drafted the formal clarification letter to the course organiser (see Appendix, `docs/COMPLAINT_LETTER.md`) — a non-technical-audience artefact the whole team reviewed and signed, and the piece of writing that taught me most about register, because my first three drafts were in my voice and my anger, and the fourth was in the team’s voice only after Robin cut three sentences that were “technically accurate but personally sharp.” He was right.

The uncomfortable closing realisation is this. I now see that building tools for others was my version of teamwork *because* I struggled with real-time collaboration — async handoff docs were emotional scaffolding for me as much as they were help for the recipient. My next project must invest in the harder skill of pairing, even though pairing feels less productive per hour, because the peer-development outcomes that matter most (Demetro’s cadence, Robin’s bug-shape instinct, the pilot’s minimalist instinct) happened in the few moments I *did* collaborate synchronously, not in any of the documents I wrote alone.